



Pathfinding



AI Movement

AI Movement nei videogiochi si riferisce al sistema che consente ai personaggi controllati dall'intelligenza artificiale di spostarsi all'interno del mondo di gioco ed è cruciale per creare esperienze di gioco credibili e immersive, in cui gli NPC sembrano agire in modo intelligente e coerente con il mondo virtuale.



AI Movement

Quando si tratta di muovere entità AI in un ambiente virtuale, emergono numerose sfide e non esiste una soluzione universale. L'approccio varia in base alle caratteristiche specifiche del gioco in sviluppo. Ad esempio:

- La destinazione dell'AI è un oggetto statico, come un pickup, oppure un'entità in movimento imprevedibile, come il personaggio del giocatore?
- L'AI deve vagare senza una meta precisa o seguire un percorso predefinito, come una sentinella in pattugliamento?

Inoltre, lo sviluppatore deve considerare variabili come terreni diversi, ostacoli e zone pericolose. La scelta tra un percorso più semplice o uno più rischioso ma rapido può influire significativamente sull'esperienza di gioco. Ogni scenario presenta problematiche uniche e affrontarle correttamente è essenziale per ottimizzare l'esperienza del giocatore.



AI Movement

Di conseguenza, questo movimento non si limita alla semplice traslazione, ma include comportamenti complessi come:

- **Navigazione:** Percorrere il livello seguendo percorsi ottimali, evitando ostacoli e interagendo con l'ambiente.
- **Pathfinding:** Utilizzare algoritmi per calcolare il percorso migliore verso una destinazione, tenendo conto di variabili come terreno, ostacoli e costi di percorso.
- **Adattamento dinamico:** Reagire a cambiamenti nell'ambiente, come la posizione del giocatore o nuovi ostacoli.
- **Comportamenti specifici:** Seguire obiettivi mobili, pattugliare un'area o muoversi in modo apparentemente casuale.



AI Movement

Durante il movimento si avranno degli

Input:

- Dati geometrici sullo stato del mondo
- Posizione attuale del personaggio
- Altre eventuali proprietà fisiche

Per ottenere poi degli

Output:

- Dati geometrici che rappresentano il movimento da effettuare (velocità, accelerazione, etc.)



AI Movement

Nella maggior parte dei giochi, possiamo categorizzare due principali tipi di movimento:

Kinematic:

- Velocità costante, no accelerazioni o rallentamenti
- Il cervello della nostra AI ritorna come output la nuova velocità dell'agente

Dynamic:

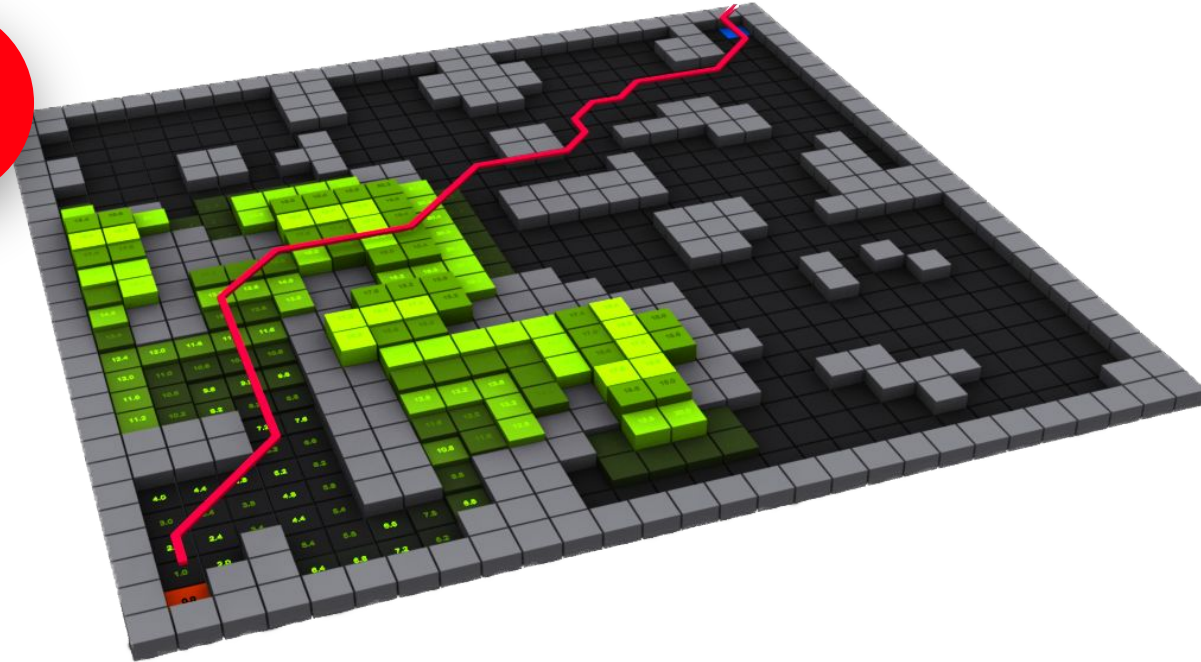
- Il movimento si calcola sulla base di sommatoria di forze, accelerazioni e decelerazioni
- Il cervello della nostra AI ritorna come output un'accelerazione da applicare all'agente per ottenere la velocità voluta



Pathfinding

Il pathfinding è una tecnica utilizzata nei videogiochi per permettere agli NPC di trovare il percorso migliore per muoversi da un punto A a un punto B all'interno del mondo di gioco.

È una componente essenziale per rendere gli ambienti interattivi e credibili, migliorando il comportamento degli agenti AI





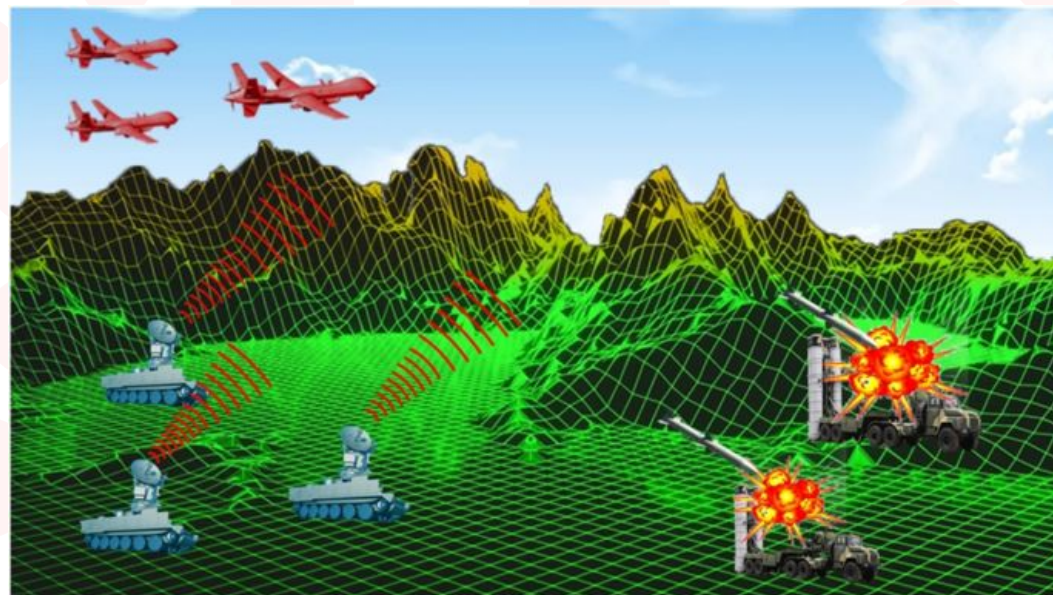
Pathfinding & Movement

Le tecniche di pathfinding genera un percorso ottimale che viene poi utilizzato dal sistema di movimento dell'AI sottostante per trasformare tali istruzioni in movimenti concreti nel mondo di gioco.

Gli algoritmi sono perciò **indipendenti** dalle capacità motorie del personaggio, in quanto calcolano il percorso basandosi su una rappresentazione astratta del mondo e non considerano direttamente le animazioni o le capacità fisiche (velocità, accelerazione, ecc.) del personaggio.

Tuttavia, le capacità motorie possono influire indirettamente sul movimento finale, influenzando indirettamente sulle scelte o sull'esecuzione del percorso, ad esempio:

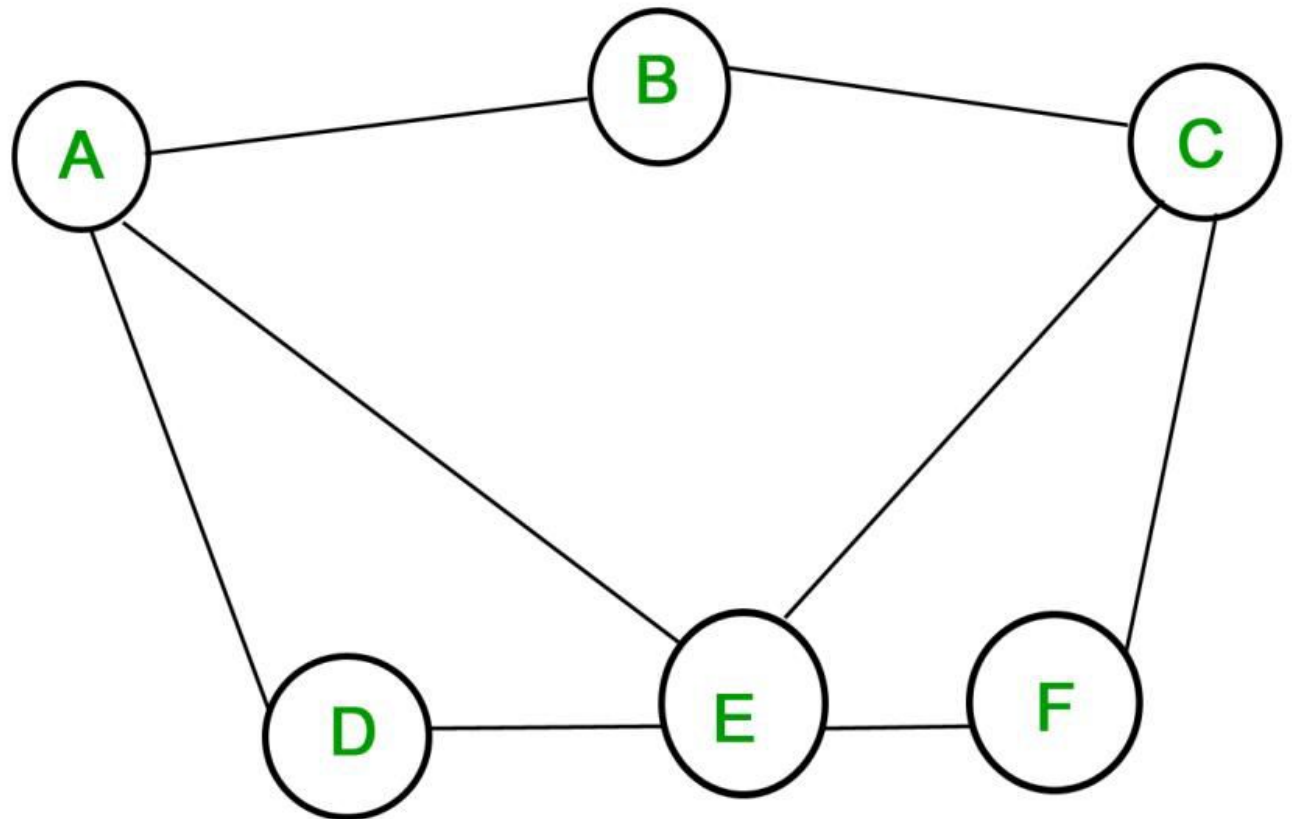
- Se un personaggio non può saltare, il sistema di pathfinding potrebbe escludere percorsi che richiedono questa abilità.
- Se un personaggio può volare, il sistema di pathfinding, invece, potrebbe aggiungere dei percorsi che sfruttano questa abilità.





Graph Theory

Ogni metodo di pathfinding ha come base una rappresentazione a **grafo**

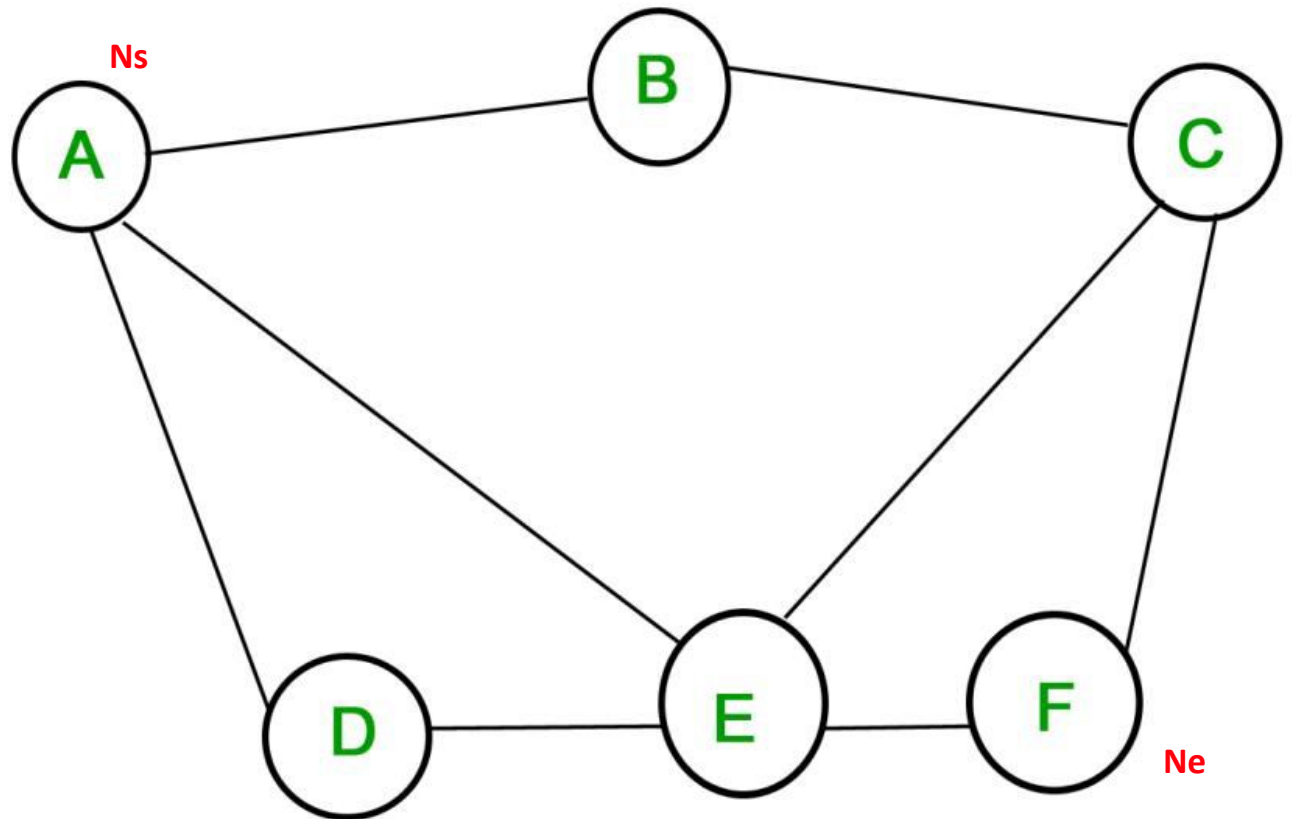




Graph Theory

Il problema di pathfinding è rappresentabile come:

- Un set di **nodi** (nodes) N
- Un set di **segmenti** (edges) E
- Un nodo di partenza **N_s**
- Un nodo di arrivo **N_e**

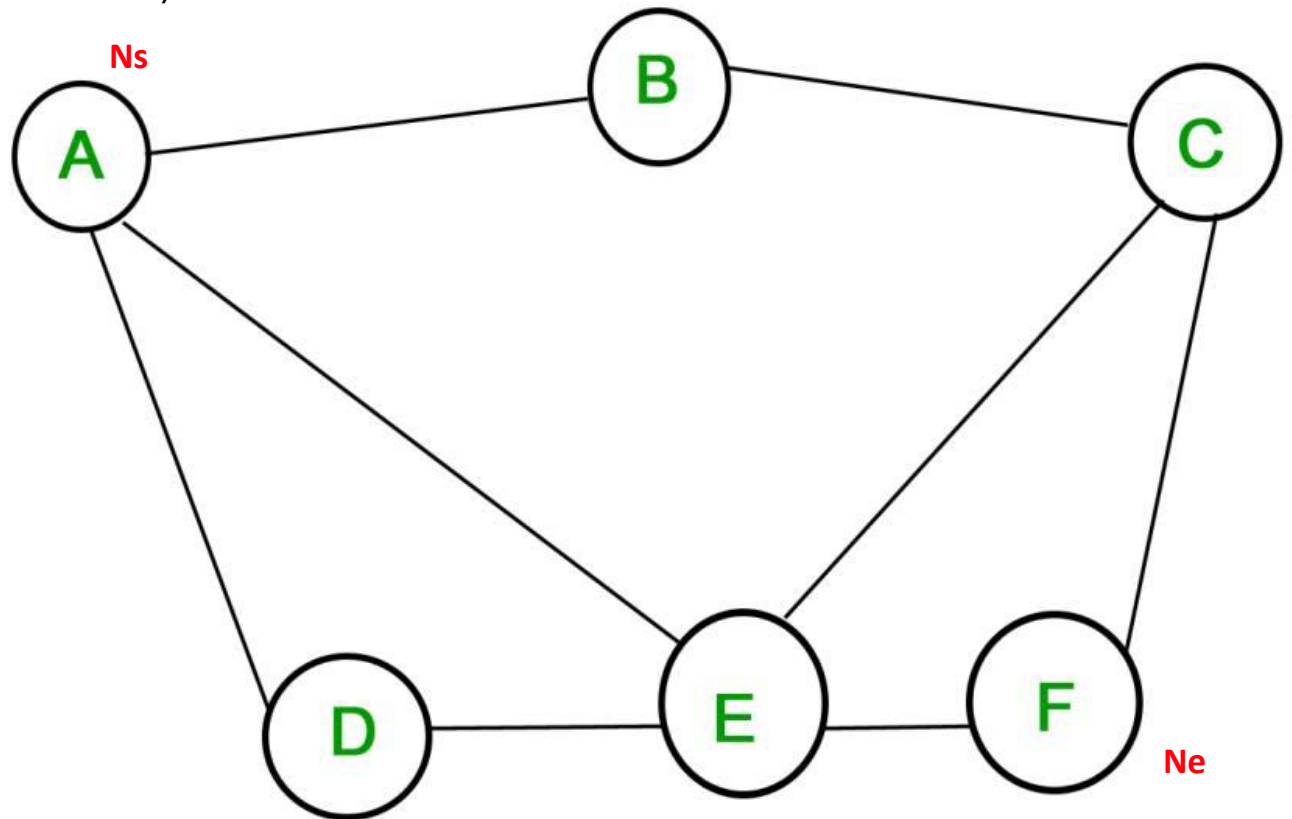




Graph Theory

Possiamo separare l'obiettivo del pathfinding in due sotto obiettivi:

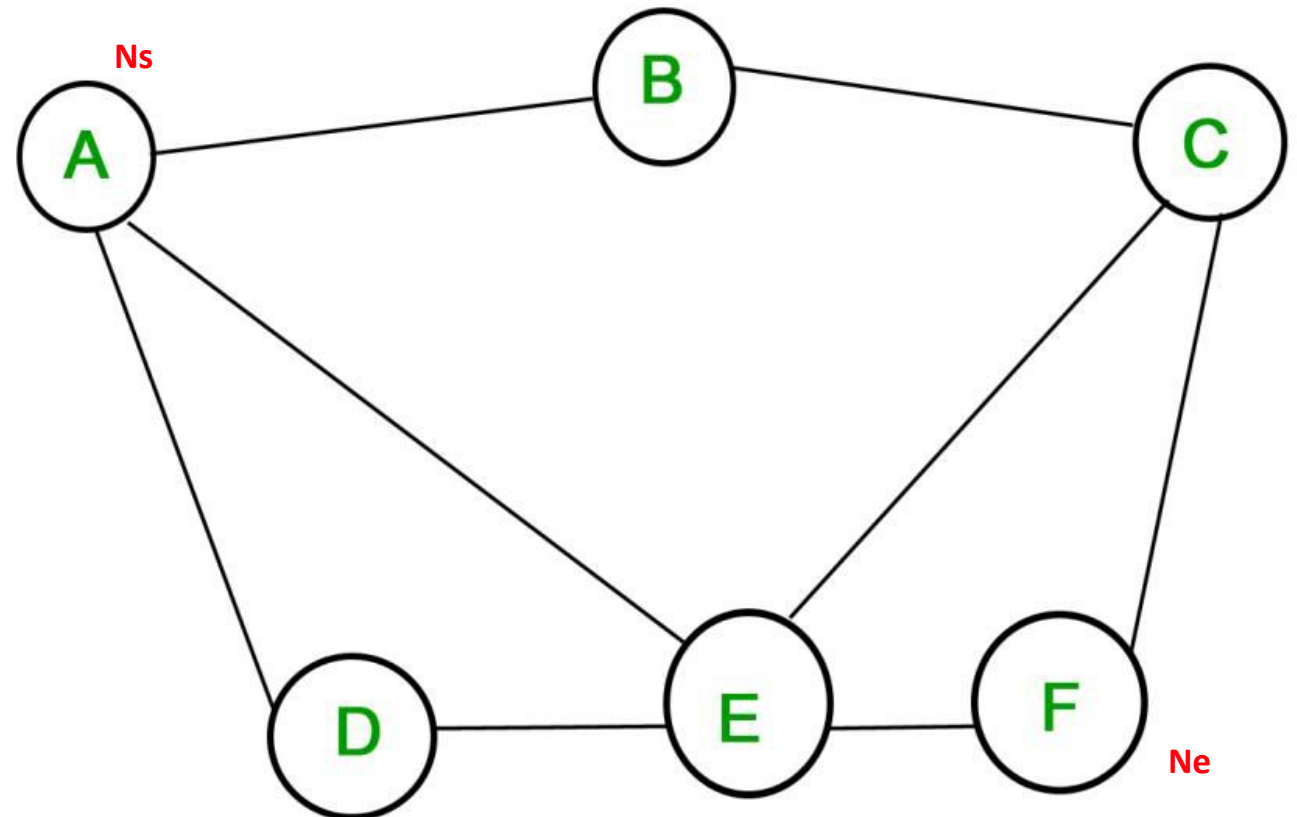
- Trovare **un percorso** tra due nodi
- Trovare il percorso **migliore** (più corto?) tra due nodi





Graph Theory

Dato il nodo di partenza, qualsiasi metodo di ricerca del grafo che lo attraversa per intero troverà un percorso (path) tra **Ns** ed **Ne**





Pathfinding Algorithms

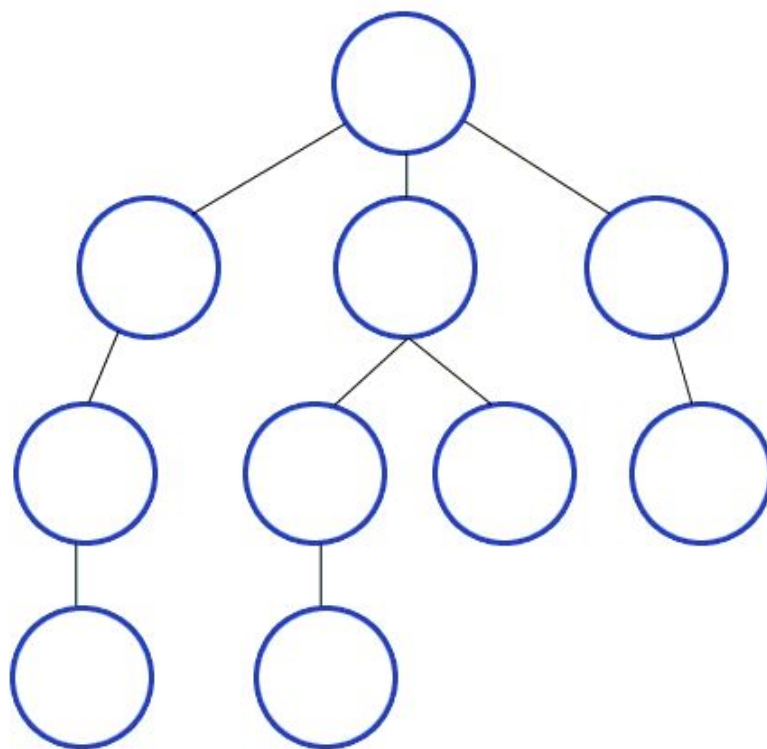
Gli algoritmi di pathfinding si basano sulla teoria dei grafi per calcolare il percorso ottimale tra due nodi del grafo.

Andiamo a vedere quelli più comuni →



Breadth - First Search

L'algoritmo Breadth - First Search (**BFS**) parte dal nodo di partenza **Ns** e cerca ogni suo nodo vicino (*neighbour*), ripetendo l'operazione finché non raggiunge il nodo finale **Ne**.





Breadth - First Search - Pseudocode

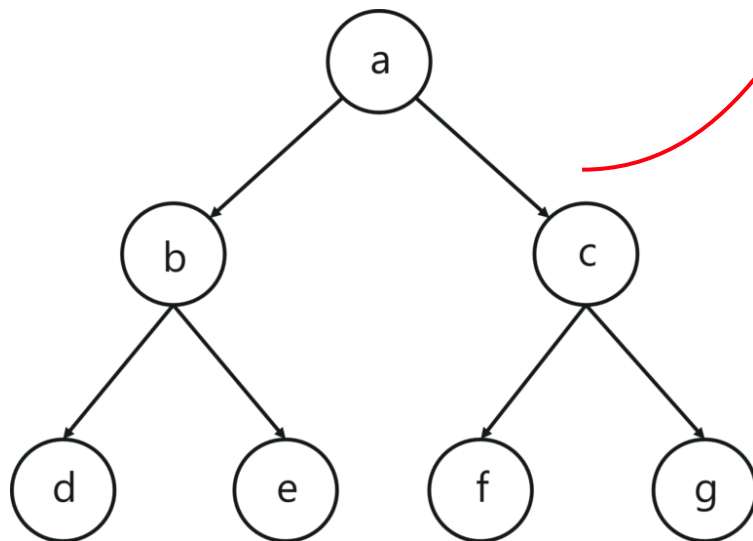
```
1 procedure BFS(G, root) is
2   let Q be a queue
3   label root as explored
4   Q.enqueue(root)
5   while Q is not empty do
6     v := Q.dequeue()
7     if v is the goal then
8       return v
9     for all edges from v to w in G.adjacentEdges(v) do
10      if w is not labeled as explored then
11        label w as explored
12        w.parent := v
13        Q.enqueue(w)
```

1. **Input (*G*, *root*)**, dove ***G*** è il grafo e ***root*** è il nodo radice.
2. Si crea una queue ***Q***.
3. Si marca il nodo ***root*** e si inizializza ***Q*** con esso.
4. Si estrae il nodo ***v*** dalla coda di ***Q*** e si valuta se è il nodo finale, altrimenti si visitano i nodi figli.
5. Si elaborano tutti i nodi figli di ***v***.
6. Si memorizzano i nodi figli ***w*** in ***Q*** per visitare successivamente i loro figli.
7. Si continua e si riparte dal punto **3** finché ***Q*** non è vuota.

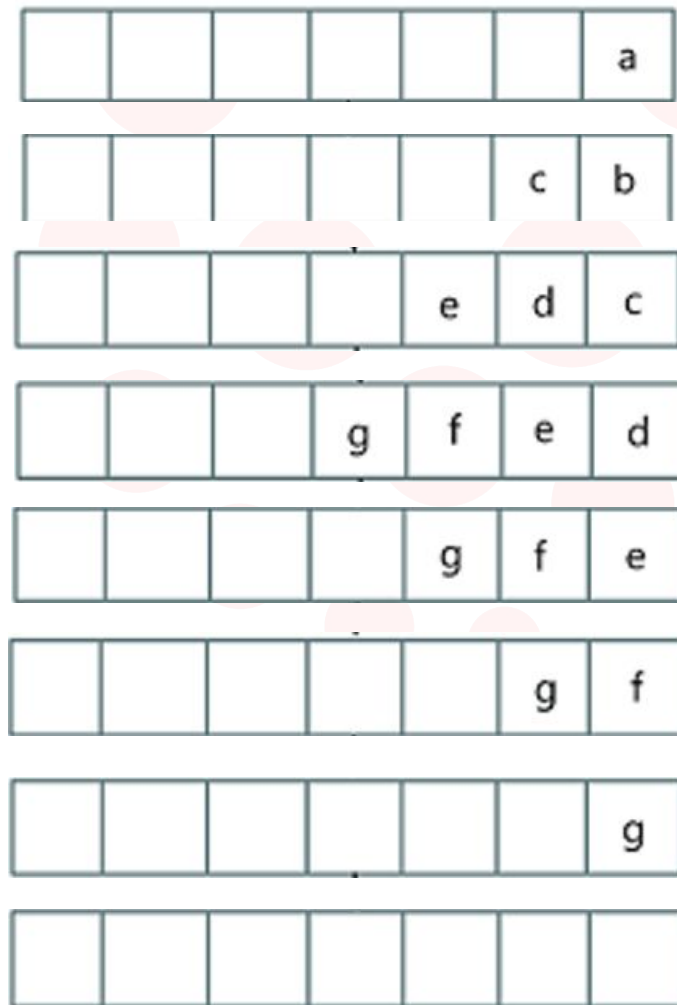


Breadth - First Search - Esempio

```
1 procedure BFS( $G$ ,  $root$ ) is
2   let  $Q$  be a queue
3   label  $root$  as explored
4    $Q.enqueue(root)$ 
5   while  $Q$  is not empty do
6      $v := Q.dequeue()$ 
7     if  $v$  is the goal then
8       return  $v$ 
9     for all edges from  $v$  to  $w$  in  $G.adjacentEdges(v)$  do
10      if  $w$  is not labeled as explored then
11        label  $w$  as explored
12         $w.parent := v$ 
13         $Q.enqueue(w)$ 
```



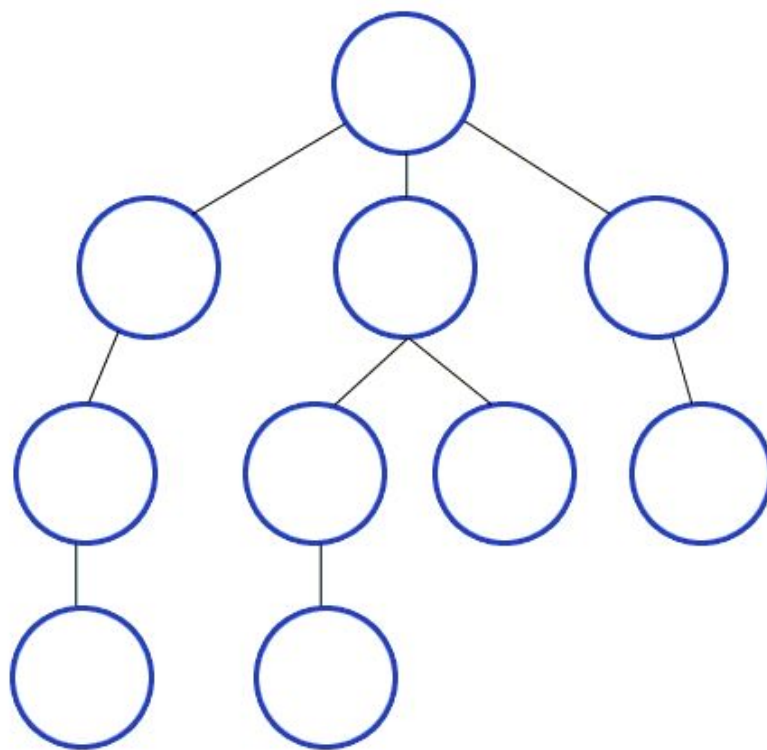
Queue





Depth - First Search

L'algoritmo Depth - First Search (**DFS**) parte dal nodo di partenza **Ns** e cerca nel primo nodo vicino (*neighbour*), ripetendo l'operazione finché non raggiunge il nodo finale **Ne**.





Depth - First Search - Pseudocode

Pseudocode iterativo

```
procedure DFS_iterative( $G, v$ ) is
  let  $S$  be a stack
   $S.push(v)$ 
  while  $S$  is not empty do
     $v = S.pop()$ 
    if  $v$  is not labeled as discovered then
      label  $v$  as discovered
      for all edges from  $v$  to  $w$  in  $G.adjacentEdges(v)$  do
        if  $w$  is not labeled as discovered then
           $S.push(w)$ 
```



Pseudocode ricorsivo

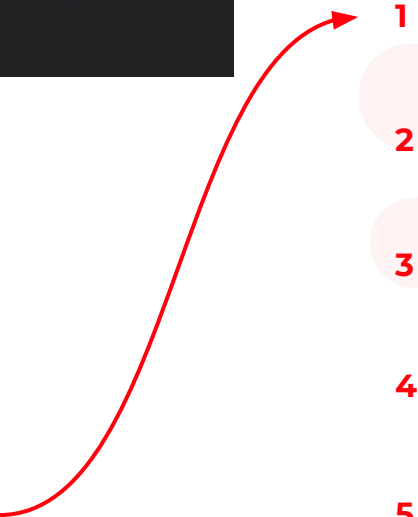
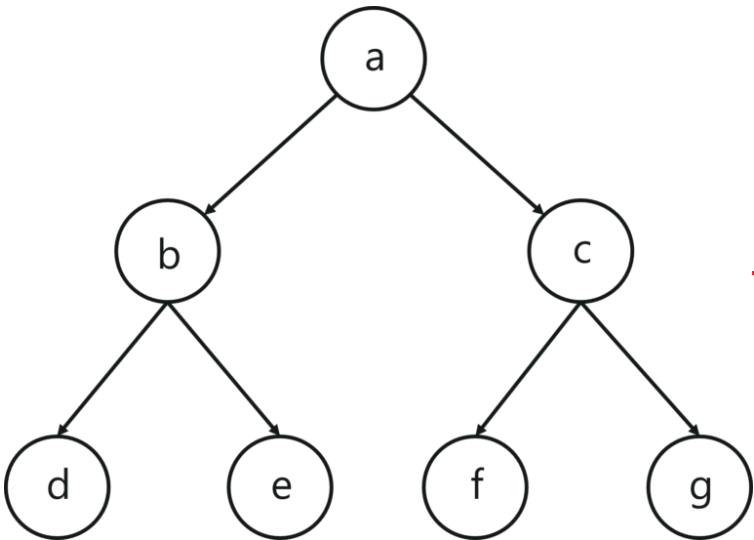
```
procedure DFS( $G, v$ ) is
  label  $v$  as discovered
  for all directed edges from  $v$  to  $w$  that are in  $G.adjacentEdges(v)$  do
    if vertex  $w$  is not labeled as discovered then
      recursively call DFS( $G, w$ )
```

1. **Input** (G, v), dove G è il grafo e v è il nodo radice.
2. Si crea uno stack S e la si inizializza con il nodo sorgente v .
3. Si estrae da S il primo nodo v dalla coda.
4. v viene marcato se non lo era già precedentemente
5. Si itera su tutti i nodi figli di v .
6. Se i nodi figli w non sono marcati allora vengono inseriti nello stack S .
7. Si continua e si riparte dal punto 3 finché S non è vuota.



Depth - First Search - Esempio

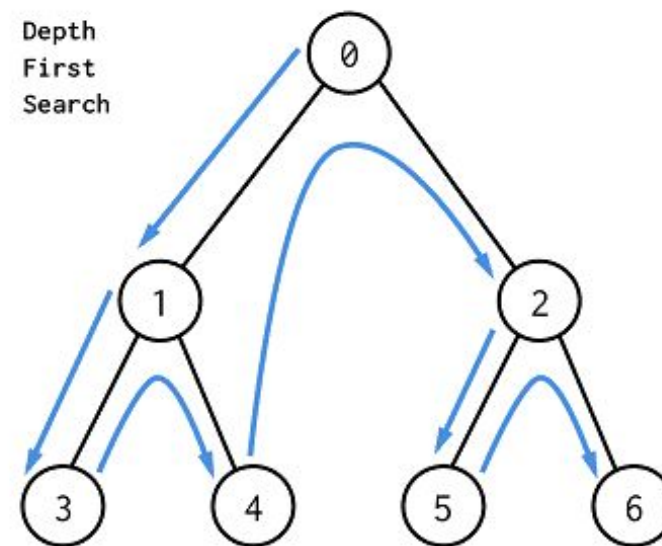
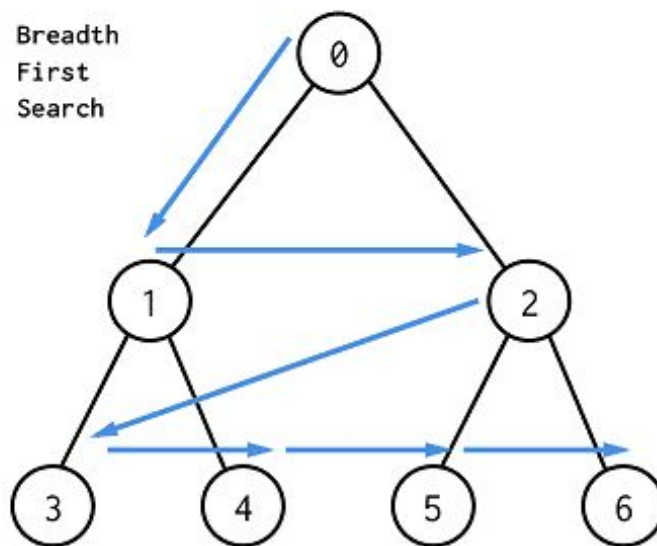
```
procedure DFS(G, v) is
  label v as discovered
  for all directed edges from v to w that are in G.adjacentEdges(v) do
    if vertex w is not labeled as discovered then
      recursively call DFS(G, w)
```



	Stack						
1	a						
2	a	b					
3	a	b	d				
4	a	b	d	e			
5	a	b	d	e	c		
6	a	b	d	e	c	f	
7	a	b	d	e	c	f	g
8							



BFS e DFS - Confronto



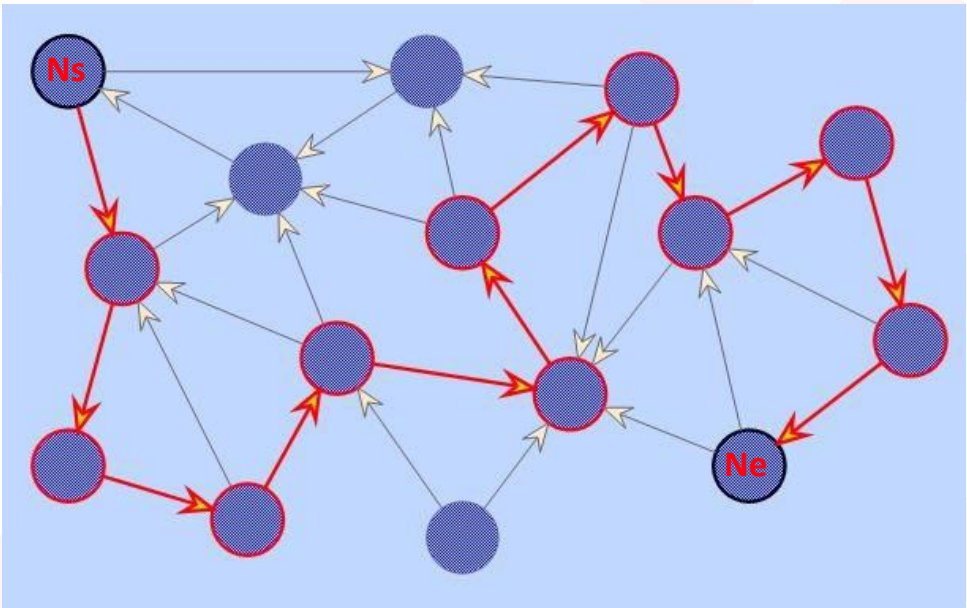
Entrambi gli algoritmi hanno una complessità lineare rispetto al numero di nodi e di segmenti.

Nodi N	Segmenti E	Percorso N+E
4	4	8
10	4	14
4	10	14
10	10	20
50	50	100
500	500	1000



Ricerca del percorso più breve

E se volessimo cercare il percorso più breve?



Dovremmo controllare **tutti** i possibili percorsi da **Ns** ad **Ne** e valutare quale sia il migliore.
Di conseguenza, la complessità diventa lineare rispetto al numero di nodi moltiplicato per il numero di segmenti.

Nodi	Segmenti	Percorso	Percorso più breve N*E
4	4	8	16
10	4	14	40
4	10	14	40
10	10	20	100
50	50	100	2500
500	500	1000	500*500 = 250000



Ricerca del percorso più breve

E' possibile eliminare strategicamente alcuni percorsi per diminuire notevolmente la complessità dell'algoritmo, raggiungendo una complessità più bassa:

Numero di segmenti moltiplicato per il logaritmo del numero dei nodi.

Nodi	Segmenti	Brute force	Strategico!
4	4	16	2.4
10	4	40	6
4	10	40	4
10	10	100	10
50	50	2500	$84 = E * \log(N)$
500	500	250000	1349

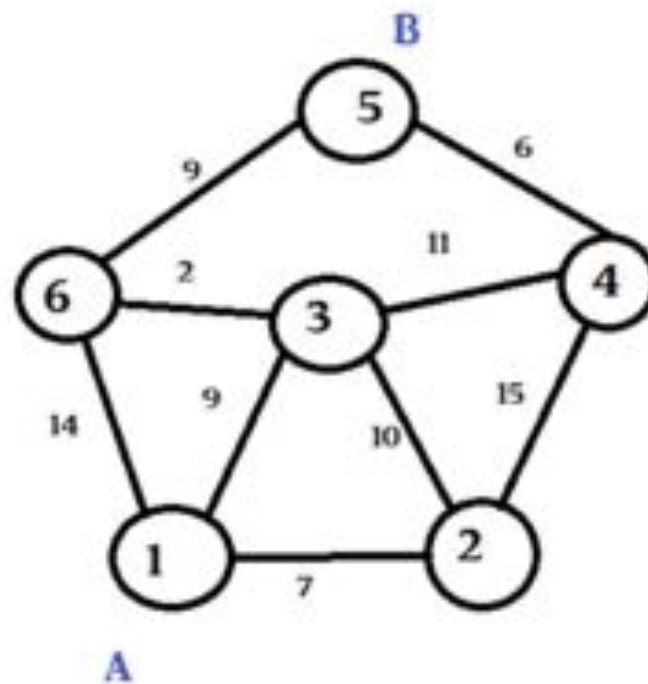
Andiamo a scoprire l'algoritmo che utilizza questa complessità →



Dijkstra

L'algoritmo di Dijkstra è un metodo per trovare il percorso più breve da un nodo sorgente a tutti gli altri nodi in un grafo **pesato** con costi non negativi (ogni segmento ha un peso ≥ 0).

Utilizza una coda di priorità per esplorare i nodi con il costo cumulativo più basso, aggiornando progressivamente le distanze minime finché non ha determinato il percorso ottimale per ogni nodo raggiungibile.





Dijkstra - Pseudocodice

```
1 function Dijkstra(Graph, source):
2
3   for each vertex  $v$  in Graph.Vertices:
4      $\text{dist}[v] \leftarrow \text{INFINITY}$ 
5      $\text{prev}[v] \leftarrow \text{UNDEFINED}$ 
6     add  $v$  to  $Q$ 
7    $\text{dist}[\text{source}] \leftarrow 0$ 
8
9   while  $Q$  is not empty:
10     $u \leftarrow$  vertex in  $Q$  with minimum  $\text{dist}[u]$ 
11    remove  $u$  from  $Q$ 
12
13    for each neighbor  $v$  of  $u$  still in  $Q$ :
14       $\text{alt} \leftarrow \text{dist}[u] + \text{Graph.Edges}(u, v)$ 
15      if  $\text{alt} < \text{dist}[v]$ :
16         $\text{dist}[v] \leftarrow \text{alt}$ 
17         $\text{prev}[v] \leftarrow u$ 
18
19   return  $\text{dist}[], \text{prev}[]$ 
```



```
1  $S \leftarrow$  empty sequence
2  $u \leftarrow \text{target}$ 
3 if  $\text{prev}[u]$  is defined or  $u = \text{source}$ :
4   while  $u$  is defined:
5     insert  $u$  at the beginning of  $S$ 
6      $u \leftarrow \text{prev}[u]$ 
```

1. **Input (G , source)**, dove G è il grafo e source è il nodo radice.
2. Si crea un set Q .
3. Ogni nodo v del grafo viene inizializzato con distanza **infinita** e successivamente aggiunto a Q .
4. Al nodo di partenza **source** viene dato valore di distanza 0.
5. Si itera fino a che esistono nodi nel set Q estraendo il nodo u con distanza minima.
6. Per ogni nodo vicino v di u ci salviamo un valore **alt** uguale al valore della distanza di u + il peso del segmento **(u, v)**.
7. Se il valore è inferiore al valore attuale della distanza di v , allora lo sostituiamo con **alt** e aggiorniamo la lista di nodi predecessori (**prev**) di v , in modo da mantenere lo storico.
8. Una volta svuotato il set, si può trovare il percorso più breve da **source** al nodo destinazione (**dest**) iterando inversamente sulla lista **prev** partendo dal nodo **dest**

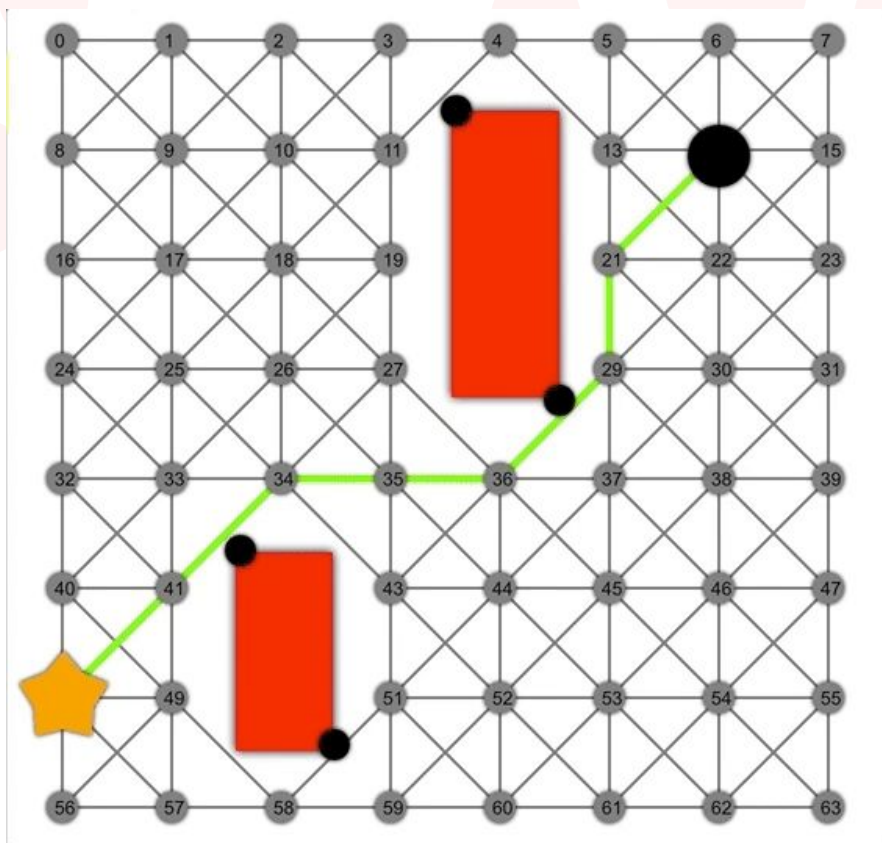


A*

L'algoritmo A* è una variante dell'algoritmo di Dijkstra utilizzata comunemente nei videogiochi.

A* aggiunge **un valore euristico** come stima della distanza tra il nodo stesso e il nodo finale, in modo da ridurre il numero di nodi esplorati in quanto esplora solo i nodi *"necessari"*.

In casi ottimali, A* è più veloce, ma richiede una buona euristica.





A* - Pseudocodice

```
1  A*(Graph, start, goal):
2      1. Initialize openSet with the start node
3      2. Create gScore map, set gScore[start] = 0 (cost from start to start is 0)
4      3. Create fScore map, set fScore[start] = h(start, goal) (heuristic estimate)
5      4. While openSet is not empty:
6          4.1 current = node in openSet with the lowest fScore value
7          4.2 If current == goal:
8              Return reconstructPath(cameFrom, current)
9          4.3 Remove current from openSet
10         4.4 For each neighbor of current:
11             4.4.1 tentative_gScore = gScore[current] + cost(current, neighbor)
12             4.4.2 If tentative_gScore < gScore[neighbor] (or neighbor is not in gScore):
13                 cameFrom[neighbor] = current
14                 gScore[neighbor] = tentative_gScore
15                 fScore[neighbor] = gScore[neighbor] + h(neighbor, goal)
16                 If neighbor is not in openSet:
17                     Add neighbor to openSet
18     5. Return "No Path Found"
```



```
1  ReconstructPath(cameFrom, current):
2      1. totalPath = [current]
3      2. While current is in cameFrom:
4          2.1 current = cameFrom[current]
5          2.2 Add current to totalPath
6      3. Return totalPath reversed
```

In modo simile all'algoritmo di Dijkstra, anche nell'A* il punto di partenza ha valore **0** e tutti gli altri nodi **infinito**.

Successivamente, sempre come Dijkstra, si itera su ogni nodo del set partendo da quello con valore minore.

Se il nodo è la nostra destinazione possiamo usare la funzione di ricostruzione del path, altrimenti continuiamo fino a trovarlo, esaminando ogni nodo adiacente aggiunto all'open set, il cui valore sarà composto dal valore del nodo stesso + la **distanza euristica dal nodo finale**.



A* - Euristicica

La funzione euristica in A* è una stima del costo per arrivare all'obiettivo a partire da un nodo.

Per ottenere una funzione euristica, denotata come **$h(n)$** , essa deve essere:

- **Ammissibile**: non deve mai sovrastimare il costo reale per arrivare all'obiettivo, per garantire la soluzione ottimale.
- **Consistente** (monotona): per ogni arco che collega due nodi n e m , vale la relazione

$$\underline{h(n) \leq c(n,m) + h(m)}$$

dove $c(n,m)$ è il costo di muoversi da n ad m .

Questa implica che il costo stimato per arrivare all'obiettivo dal nodo n non è mai maggiore del costo di spostarsi verso un vicino m + il costo stimato per arrivare all'obiettivo da m .

Le euristiche comuni sono:

- Distanza Euclidea: per movimenti liberi in uno spazio continuo.
- Distanza di Manhattan: per spostamenti solo orizzontali/verticali.
- Distanza a passi: per percorsi con ostacoli.

L'euristica deve essere scelta in base alla struttura del problema e deve essere ammissibile per assicurare un'ottima ricerca.



A* - Euristicica

Come ottenere una funzione euristica?

- **Analizzare il problema specifico:** Prima di scegliere un'euristica, devi comprendere la struttura del problema. È importante conoscere come si può misurare la "distanza" tra il nodo attuale e l'obiettivo.
- **Garantire l'ammissibilità:** Assicurati che la funzione euristica non sovrastimi mai il costo per arrivare all'obiettivo. Se sovrastima, A* non troverà la soluzione ottimale.
- **Selezionare una funzione che rifletta il tipo di movimento possibile:** Esistono delle euristiche comuni in base al tipo di movimento previsto.

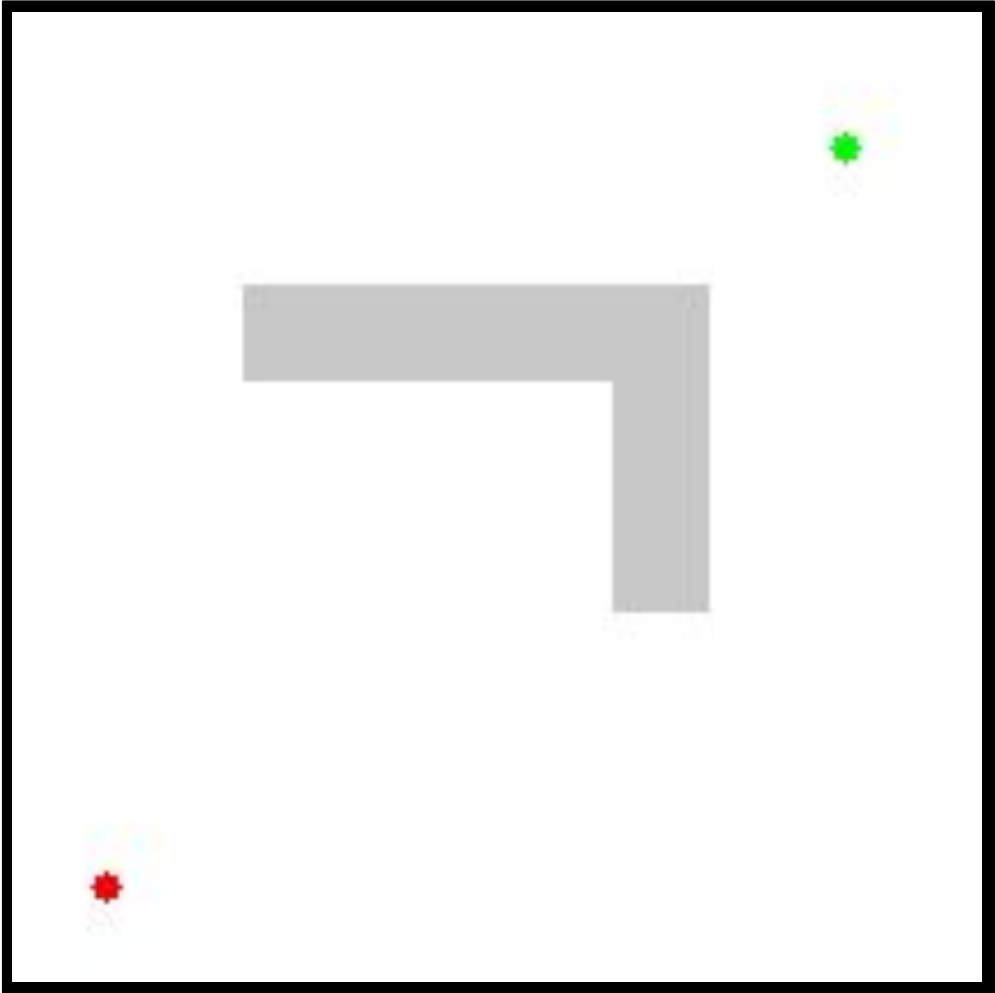
Esempi:

- Distanza Euclidea: se gli stati sono punti in uno **spazio continuo**, la distanza euclidea tra il nodo corrente e l'obiettivo può essere una buona euristica (specialmente per il caso di navigazione in un piano).
- Distanza di Manhattan: in griglie dove è possibile muoversi **solo in orizzontale e verticale**, la distanza di Manhattan (somma delle differenze assolute tra le coordinate) è una funzione euristica ammissibile.
- Distanza a passi: se **ci sono ostacoli** o altri vincoli, si può utilizzare la distanza a passi, che conta il numero minimo di movimenti richiesti.

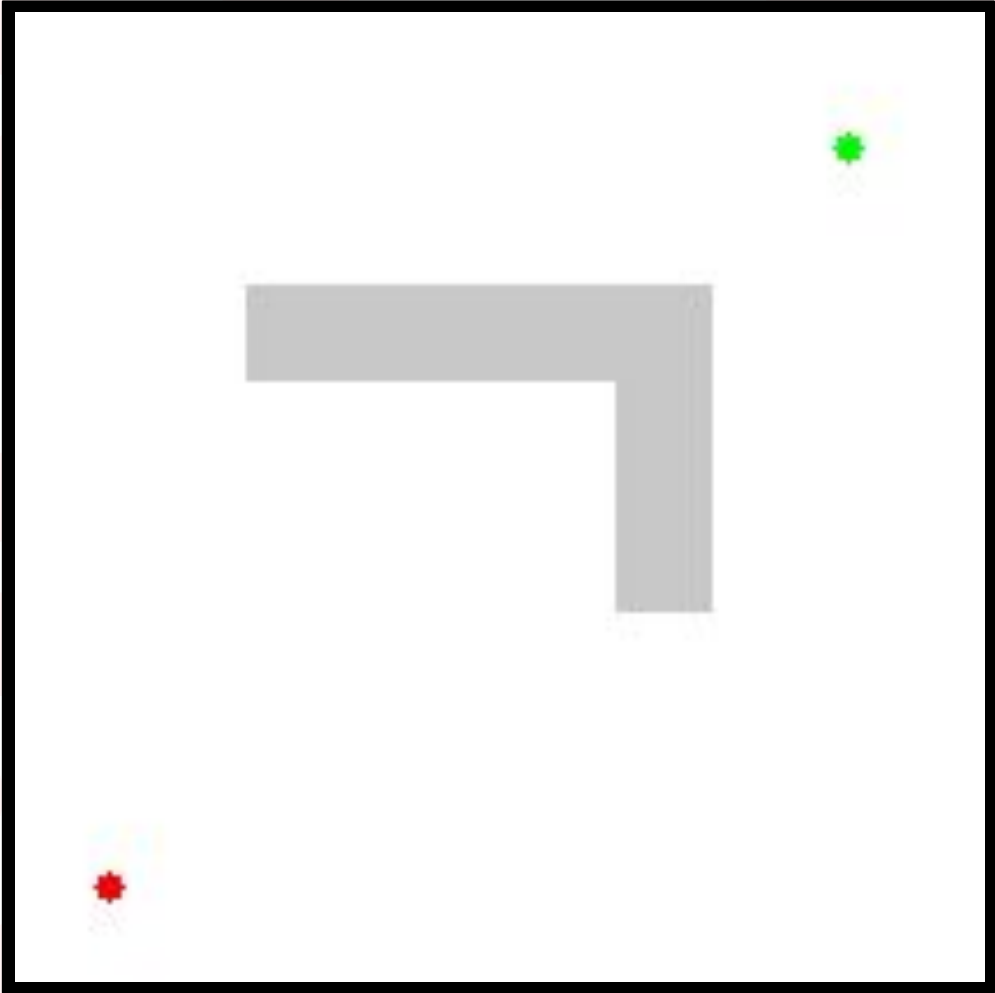


Confronto

Dijkstra



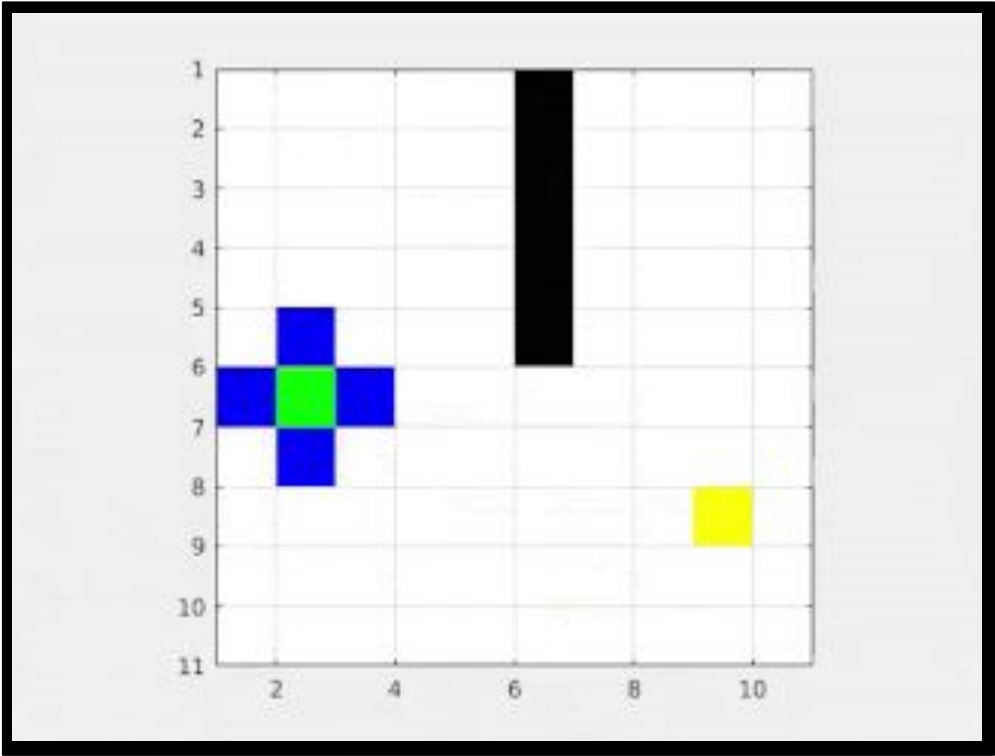
A*





Confronto - 2

Dijkstra



A*

